Submodule 33.08: High-Dimensional Data Visualization (PCA 2D vs 3D Projections)

Topic Overview & Dimensionality Reduction

Our customer transaction dataset now resides in an 18-dimensional continuous vector space ($\mathbb{R}^{18}$). Humans can visually conceptualize spatial relationships in at most 3 dimensions.

To visualize customer clusters and denoise our coordinate space before clustering, we employ **Principal Component Analysis (PCA)**. PCA identifies orthogonal axes (eigenvectors of the covariance matrix) that maximize the variance of projected customer data:

$$\Sigma = \frac{1}{n} X^T X$$

$$\Sigma v_k = \lambda_k v_k$$

where λ_k represents the eigenvalue (variance captured by the k-th principal component v_k).

---

1. Explained Variance: 2D vs 3D Projections

Reference: smartcart.ipynb (Cells 40 - 45).

2D Projection Trade-off:

- PC_1: Captures **23.16%** of variance.
- PC_2: Captures **11.39%** of variance.
- Cumulative Variance (2D): $23.16\% + 11.39\% = \mathbf{34.55\%}$.
- *Limitation*: Compressing 18 dimensions down to 2 discards over 65% of customer variance, collapsing distinct sub-clusters into single overlapping visual blobs.

3D Projection Optimization:

- PC_3: Captures **10.41%** of variance.
- Cumulative Variance (3D): $23.16\% + 11.39\% + 10.41\% = \mathbf{44.95\%} \approx 45\%$.

- Adding the 3rd principal component increases explained variance by almost a third, exposing depth and separating overlapping customer clouds.

---

2. Python Implementation (3D PCA Scatter Plot)

Reference: smartcart.ipynb (Cells 43 - 45).

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Fit 3-Component PCA
pca = PCA(n_components=3, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
# Output: [0.23163158, 0.11385454, 0.10405815]
print("Total Explained Variance:", pca.explained_variance_ratio_.sum())
# Output: ~0.4495 (44.95%)

# 3D Visualization
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], alpha=0.6, s=15)
ax.set_xlabel("PCA 1")
ax.set_ylabel("PCA 2")
ax.set_zlabel("PCA 3")
ax.set_title("3D PCA Customer Projection")
plt.show()
```

---

3. Key Observations & Algorithm Selection

1. **Visual Manifold Inspection**: The 3D scatter plot reveals two primary spatial clouds along PC_1 (low spenders on one side, high spenders on the other), but within each cloud, data points form a continuous distribution rather than isolated islands.
2. **Why DBSCAN is Inappropriate**: DBSCAN relies on global density parameters (ε, `min_samples`) and requires clear low-density valleys to delineate clusters. On continuous commercial customer manifolds, DBSCAN tends to fragment into 30+ tiny micro-clusters while flagging valid boundary spenders as noise.
3. **Preferred Algorithms**: Partitioning (**K-Means**) and agglomerative hierarchical merging (**AgglomerativeClustering**) are specifically designed for continuous spherical/elliptical segmenting of customer bases.